



SHAHID BEHESHTI UNIVERSITY

DATA MINING

AMIRHOSEIN AZARPOUR

404422005

Theory part

1)

a) As the value of the regularization parameter increases in ridge regression, all model coefficients move smoothly and uniformly toward zero, but none of them become exactly zero. In contrast, in the lasso method, as the same parameter increases, many coefficients become exactly zero and are removed from the model, leaving only a few of the most important ones. The reason shrinking coefficients generally improves model performance on new data is that large coefficients are often a sign of overfitting — meaning the model has memorized random noise and idiosyncrasies in the training data. By shrinking coefficients toward zero, the model becomes simpler and more general, thereby improving its predictive ability on unseen data.

b) The key geometric difference between these two methods is related to the shape of the constraint region. In ridge regression, the constraint region is a circle or sphere, whose boundary is completely smooth and curved. In lasso, however, the constraint region is a rhombus or diamond shape, which has sharp corners or tips on the axes. Therefore, when the error contour meets this region, due to the absence of corners in ridge, it is very unlikely that the intersection occurs exactly on an axis — so no coefficient becomes zero. But in lasso, because of the sharp corners on the axes, the error contour is very likely to hit the constraint region exactly at one of these corners, causing some coefficients to be exactly zero and thus effectively removed.

c) When multiple predictor variables are highly correlated with each other, ridge regression tends to distribute their coefficients relatively equally and balanced among them — meaning each of the correlated variables receives a roughly equal share of the final coefficient. In contrast, lasso tends to randomly or based on minor criteria select only one of those variables and keep its coefficient non-zero, while setting the coefficients of the other variables exactly to zero. If the main goal is model simplicity and easy interpretability, lasso is a much better choice because it produces a model with fewer variables, making it easier for humans to understand the relationship of each with the output. Ridge regression, on the other hand, keeps all variables in the model, resulting in a bulky and complex model that is more difficult to interpret.

2)

In part (a), as the classification threshold moves from zero to one, an inverse relationship between sensitivity and specificity emerges. Initially, when the threshold is very low, the model predicts almost all samples as positive, so sensitivity is very high because nearly all frauds are detected, but specificity is very low because many legitimate transactions are incorrectly classified as fraud. As the threshold increases, the model only labels a sample as positive when it has high confidence. In this case, sensitivity decreases because some actual frauds are missed, but specificity increases because legitimate transactions are identified more accurately. Finally, when the threshold reaches one, the model does not label any sample as positive, so sensitivity drops to zero and specificity becomes one. This inverse relationship is called the sensitivity-specificity trade-off, and the choice of the optimal threshold depends on the cost of each type of error in the problem.

In part (b), the area under the ROC curve (AUC) indicates how well the model can correctly rank the predicted probabilities for positive and negative samples, without being dependent on any specific threshold. In other words, if we randomly select one positive sample and one negative sample, the AUC equals the probability that the model assigns a higher score to the positive sample than to the negative one. This value lies between 0.5 and 1. The closer it is to 1, the better the model is at almost always ranking the positive sample's score higher than the negative one, indicating high discriminative power. The closer it is to 0.5, the more the model is essentially unable to distinguish between the two classes.

In part (c), if the model performs no better than random guessing, the predicted probability distributions for the positive and negative classes completely overlap and are indistinguishable. In this case, the ROC curve becomes a diagonal line from the bottom-left corner to the top-right corner of the plot — that is, a line at a 45-degree angle. The reason is that at any given threshold, the true positive rate and the false positive rate are equal because the model has no ability to distinguish the positive class from the negative class and essentially behaves like a coin toss. The area under this curve is exactly 0.5, indicating that the model has no meaningful predictive power.

3)

a) A hard-margin classifier is sensitive to every data point and does not allow any point to lie inside the margin or on the wrong side of the decision boundary. When an outlier is located inside the cluster of the opposite class, this method is forced to dramatically change its entire decision boundary in order to classify that point correctly on its own side. As a result, the margin becomes very narrow, and the decision boundary becomes distorted just to accommodate that one noisy point. This leads to poor generalization because the model has adapted to noise and exceptions instead of learning the underlying pattern of the data, and therefore it will perform poorly when new data from the true distribution arrive.

b) A soft-margin support vector machine, unlike the hard-margin version, is not highly sensitive to outliers. Geometrically, this method works by allowing the model to ignore some points and let them lie on the wrong side of the boundary or inside the margin, but it imposes a penalty for such violations. In other words, the model tries to create a relatively large margin and only allows a limited number of points to cross this margin or fall on the wrong side. When faced with an outlier located inside the opposite class cluster, the soft-margin model prefers to treat that point as an exception and let it remain where it is, without shifting the entire decision boundary just to accommodate that single point. Consequently, the final decision boundary is determined primarily by the main points and the dominant cluster, resulting in a larger and more reasonable margin.

c) Regarding the parameter C , it controls the penalty for points that violate the margin. In response to the first question: when C is very large, the model is highly sensitive to margin violations and imposes a heavy penalty, so it tries to create a narrow margin to ensure no point violates the margin. Conversely, when C is very small, the model easily overlooks margin violations and allows a very wide margin. In response to the second question: with a very large C , the number of allowed margin violations is very low, close to zero, because the model has no tolerance for errors. With a very small C , the number of allowed margin violations becomes very large, and the model allows many points to lie inside the margin or on the wrong side. In response to the third question: when C is very large, the model has high variance and low bias, and is prone to overfitting because the hard, narrow margin also learns noise. When C is very small, the model has high bias and low variance, and may suffer from underfitting because the margin is so wide that it misses much of the true data pattern. The optimal value of C is where a balance is achieved between these two extremes, so that the model has enough complexity to learn the pattern while remaining robust to noise.

4)

a) Randomly splitting the data into two equal halves for training and validation can produce very different performance estimates, because each time a specific combination of samples enters the training set and the validation set. If, by chance, simpler samples or more repetitive patterns go into the training set and more difficult samples go into the validation set, the apparent performance of the model will be weak, and vice versa. This shows that the evaluation method using a single simple split has very high variance and is unstable — meaning that if we repeat the same process with a different random seed, we may obtain a completely different performance number, and we cannot trust this point estimate.

b) Training the model on only half of the available data means that the model sees less training data than what is actually available. As a result, the estimated test error is usually overestimated compared to the true real-world error. The reason is that a model trained on less data learns fewer patterns and performs more poorly. When we later retrain the same model on all the data, it will perform better. Therefore, the simple split method typically underestimates real-world performance — that is, it shows a higher error than the true value. In other words, this method is neither optimistic nor pessimistic per se; rather, because of the small training size, it makes the model appear weaker than it actually could be.

c) When the target variable is highly imbalanced — for example, only one percent of samples are positive — the main problem is that in a random split, all or most of the rare positive samples might by chance go into the training set, leaving no positive samples in the validation set. As a result, during validation, the model would have no positives to evaluate, making metrics like sensitivity incalculable or highly unreliable. In the other scenario, all positive samples might go into the validation set, and the model would never see them during training, thus failing to learn how to detect them. Regarding extreme outliers, the problem is that if an important outlier falls into the training set, the model will adapt to it and overfit; but if that same outlier falls into the validation set, the model has never seen it and will make a very large error in predicting it, severely affecting the performance estimate. In both cases, the learning curve shows that as training sample size increases, variance decreases — but in the simple split method with a small training size, this variance is very high, and the imbalanced distribution of rare observations or outliers between the two sets reduces the reliability of the evaluation to nearly zero.

5)

Part (a): Purity of a node means that the majority, or even all, of the samples in that node belong to a single class. In other words, the higher the proportion of samples from a particular class in a node, the purer that node is, and the best case is when all samples in the node are from one class. Maximizing node purity is the main goal in building a classification tree, because a purer node provides a more reliable prediction. When the tree reaches leaves that are very pure, it can say with high confidence which class any new sample reaching that leaf belongs to. An ideal tree will eventually have leaves that are completely pure, but one must be careful not to make it too deep to avoid overfitting.

Part (b): Classification error seems like a simple measure, equal to the proportion of samples in a node that belong to the minority class, but this measure is not preferred for growing a tree because it is not sensitive enough. The conceptual difference between the Gini index and entropy on one hand, and classification error on the other, is that the first two are more sensitive to the class distribution within a node and better reflect small changes in purity. For example, if in a node we initially have three classes with proportions 45%, 35%, and 20%, and through a split we reach a node with proportions 55%, 45%, and 0%, classification error — which only looks at the majority class — changes very little, whereas Gini and entropy detect this improvement. For this reason, using classification error causes the tree to stop growing earlier, missing the opportunity to make subsequent useful splits.

Part (c): Although Gini and entropy often produce similar results in selecting the best split, for very large datasets the Gini index is preferred from a computational efficiency perspective. The reason is that calculating entropy requires computing logarithms, which is a heavy and time-consuming operation, whereas the Gini index involves only multiplication, addition, and subtraction, which are much faster to execute. Additionally, in the hint provided, it is noted that when the class proportion improves but the majority class remains the same, classification error does not change at all, whereas Gini is sensitive to these changes. This sensitivity of Gini, combined with its computational simplicity, makes it an ideal choice for large datasets.

6)

a) The conceptual goal of PCA is to reduce the dimensionality of the data while retaining the maximum amount of information — or more precisely, the maximum amount of variation present in the data. The geometric meaning of the first principal component is that it finds a straight line in the data space that shows the greatest elongation or spread of the data cloud. If you imagine a cloud of points that is elongated and stretched, the first principal component is the line that runs along the length of this cloud. This component is typically rotated or skewed relative to the original feature axes and is considered a linear combination of all the original features, meaning that each new axis is a weighted sum of the old axes.

b) If one feature is measured in millimeters and another in kilometers, and we apply PCA without standardization, the principal components will be heavily influenced by the feature with the larger scale — that is, the one measured in kilometers. In other words, the kilometer feature, because of its large numerical values, will have much higher variance than the millimeter feature, and PCA, in order to maximize variance, will assign nearly all the weight to that feature while ignoring the millimeter feature. This undermines the goal of PCA, because PCA is supposed to find the true underlying pattern of the data, not be merely influenced by the units of measurement. An important feature measured on a small scale might contain a critical pattern, but it gets ignored due to its small numerical magnitude, while an unimportant feature with a large scale dominates everything.

c) A key geometric limitation of standard PCA is that it can only discover linear structure in the data and cannot properly capture nonlinear manifolds. If the data lie on a horseshoe shape or a Swiss roll, PCA cannot effectively reduce the dimensionality while preserving the true relationships between points, because PCA tries to model variation with straight, flat lines. In a nonlinear manifold, the true proximity between points is defined along a curved surface, not along a straight line. For example, on a horseshoe, two points that are close to each other along the curved distance of the horseshoe may actually be far apart in the original space when measured along a straight line. PCA, which only looks for straight lines of high variance, will treat these points as far apart and lose the intrinsic manifold structure. For such data, nonlinear dimensionality reduction methods such as nonlinear multidimensional scaling are more appropriate.